

Sicurezza

CdL in Informatica – L31

Lezione L7 Programmi e Programmazione (Malware)

Prof. Emiliano Casalicchio
Department of Computer Science
Sapienza University of Rome



SAPIENZA
UNIVERSITÀ DI ROMA



Malicious Code - Malware

Capitolo 3 – Sezione 3.2



Type of Malware

Virus

- a program that can replicate itself and pass on malicious code to other nonmalicious programs by modifying them
 - A **transient virus** has a life span that depends on the life of its host (run when the host run)
 - A **resident virus** locates itself in memory; will terminate when the system Alt
- The spreading can leverage any medium
- Cannot run and replicate without an host

Worm

- a program that spreads copies of itself through a network
- is standalone, does not need an host



Type of Malware (cont'd)

Trojan Horse

- malicious code that, in addition to its primary effect, has a second, nonobvious, malicious effect
 - E.g. a login script that passes the identification information on to the rest of the system for login processing, but also retains a copy of the information for later, malicious use



Other type of Malware

Worm and Virus define the way a malicious program can propagate and execute, then there are many other code name for specific malicious actions (the payload), e.g.

Rabbit

Code that replicates itself without limit, to exhaust resources

Logic bomb

Code that triggers action when a predetermined condition occurs

Time bomb

Code that triggers action when a predetermined time occurs

Dropper

Transfer agent code only to drop other malicious code, such as virus or Trojan horse

Script attack, JavaScript, Active code attack

Malicious code communicated in JavaScript, ActiveX, or another scripting language, downloaded as part of displaying a webpage

RAT (remote access Trojan)

Trojan horse that, once planted, gives access from remote location

Spyware

Program that intercepts and covertly communicates data on the user or the user's activity

Ransomware

Attack that transfers data offsite or, more usually, encrypts it; the attacker demands a ransom in exchange for returning the data or providing the decryption key



Other type of Malware (Cont'd)

Bot	Semi-autonomous agent, under control of a (usually remote) controller or “herder”; not necessarily malicious	Trapdoor or backdoor	Code feature that allows unauthorized access to a machine or program; bypasses normal access control and authentication
Zombie	Code or entire computer under control of a (usually remote) program	Tool or toolkit	Program containing a set of tests for vulnerabilities; not dangerous itself, but each successful test identifies a vulnerable host and means of attack
Browser hijacker	Code that changes browser settings, disallows access to certain sites, or redirects browser to other sites	Scareware	Not code; false warning of malicious code attack
Rootkit	Code installed in “root” or most privileged section of operating system; hard to detect		



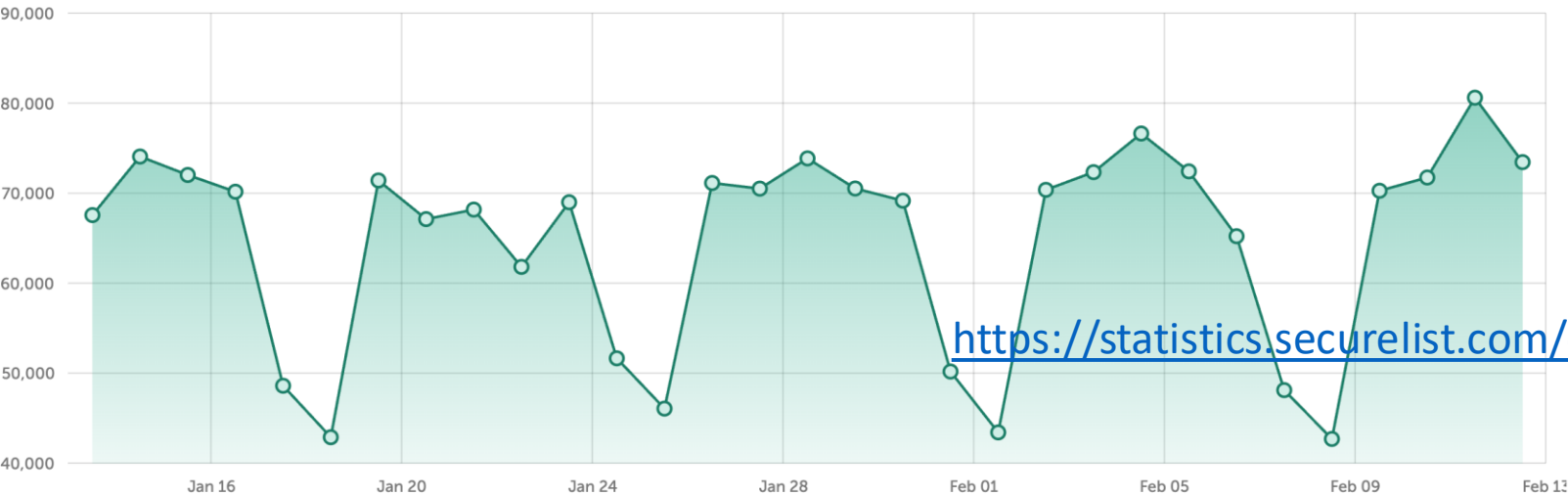
Italian Republic

RansomwareExploitsWeb threatsSpamMalicious mailNetwork attacksLocal InfectionsOn-demand scan

DayWeekMonth

Statistics on threats detected on the devices of Kaspersky users.

Number of notifications



<https://statistics.securelist.com/country/italy/on-access-scan/month>

Top 10 detected threats (verdicts issued by the security solution) for month.

1	Trojan.NSIS.Agent.teup	19.76%
2	DangerousObject.Multi.Generic	7.77%
3	Backdoor.MSIL.ConvertRat.gen	6.20%
4	Trojan.Script.Generic	4.42%
5	Trojan.Win64.Agentb.lgnu	3.84%

We are interested in exploring four aspects of malicious code infections:

- ***harm***—how they affect users and systems
- ***transmission and propagation***—how they are transmitted and replicate, and how they cause further transmission
- ***activation***—how they gain control and install themselves so that they can reactivate
- ***stealth***—how they hide to avoid detection



Harm from Malicious Code

Three categories of payload

- **Nondestructive**
 - disturb, create panic
- **Destructive**
 - corrupts files, deletes files, damages software, or executes commands to cause hardware stress or breakage
- **Commercial or criminal intent**
 - installing code to allow a remote agent to cause the computer to perform actions on the agent's signal or to forward sensitive data to the agent.
 - E.g. collecting personal data (for example, login credentials to a banking website), collecting proprietary data, or serving as a compromised agent for sending spam email or mounting a denial-of-service attack



Nondestructive harms

Hiding the cursor

Displaying text or an image on the screen

Opening a browser window to websites related to current activity

- e.g., opening an airline webpage when the current site is a foreign city's tourist board

Sending email to some or all entries in the user's contacts or alias list.

- the email would be delivered as having come from the user, leading the recipient to think it authentic.
- the Melissa virus did this, sending copies of itself as an attachment that unsuspecting recipients would open, which then infected the recipients and allowed the infection to spread to their contacts.



Destructive harms

Opening text documents and changing some instances of “is” to “is not,” and vice versa.

- The malware may change only a few instances in random locations, so the change would not be readily apparent.
- Imagine the effect these changes would have on a term paper, proposal, contract, or news story.

Deleting all files

- The Jerusalem virus did this every Friday that was a 13th day of the month.
- A variation on this theme is ransomware, in which the attacker encrypts the victim's files and offers to supply the decryption key only after a substantial ransom is paid.

Modifying system program files

- to ensure subsequent reactivation and avoid detection.

Modifying system information

- such as the Windows registry (the table of all critical system information).



Commercial or criminal intent harms

- Stealing and forwarding sensitive information such as passwords and login details.



Harm to the User's System

Malware code is written in a way that resists attempts to eradicate it.

Example of techniques to conceal infection and complicate detection and eradication

- Hide the file in a lower-level directory
- Attach malware to a critical system file, especially one that is invoked during system startup (to ensure the malware is reactivated)
- Replace (retaining the name of) a noncritical system file
- Hide copies of the executable code in more than one location
- Hide copies of the executable in different locations on different systems so no single eradication procedure can work easily
- Modify the system registry so that the malware is always executed or malware detection is disabled



Ridding a system of malware can be difficult

- The infection can be in many places
 - the system area, installed programs, the user's data, or undocumented free space.
- Copies can move back and forth between memory and storage
 - after one location is cleaned, the infection is reinserted from the other location.
- Viruses could have a **multipartite** form
 - they install themselves in several pieces in distinct locations, sometimes to carry out different objectives
 - if only one piece is removed, the remaining pieces can reconstitute and reinstall the deleted piece

Eradication requires destroying all pieces of the infection

For more deeply established infections, users may have to erase and reformat an entire disk



Harm to the World

- A malware can spread to millions of systems
 - “I Love You” 10M
 - Code Red 3M
 - Conficker 1.4M
- Cost of recovery > millions of dollars
- Cost is estimated as the sum of the damaged systems
 - Cost of hw, of data, of manpower to recover; bussiness revenue lost; image



We are interested in exploring four aspects of malicious code infections:

- *harm*—how they affect users and systems
- ***transmission and propagation***—how they are transmitted and replicate, and how they cause further transmission
- *activation*—how they gain control and install themselves so that they can reactivate
- *stealth*—how they hide to avoid detection



Transmission and Propagation

Once stored on the disk, the malware need activation to start spreading, i.e. the malware must be executed.

Techniques are many

- Setup and Installer Program Transmission
- Attached File
- Document Viruses
- Autorun
- Propagation
- Appended viruses
- Viruses That Surround a Program
- Integrated Viruses and Replacements



Setup and Installer Program Transmission

- Virus code inserted in
 - a program on the distribution medium
 - downloaded in the installation package
- When executed (the program or installer) the virus could install itself
 - on a permanent storage medium (typically, a hard disk)
 - in any and all executing programs in memory
- Human intervention is necessary to start the process
 - to puts the virus on the distribution medium/installer
 - to execution the program to which the virus is attached
- After that, infection spread without human intervention



Attached File transmission

Common means of virus activation

The virus is in a file attached to a message or embedded in another file, such as one to be downloaded from a website

- The virus writer tries to convince the victim (the recipient of the message or file) to open the object
- Once the viral object is opened (and thereby executed), the activated virus can do its work
- The virus can be executable code embedded in an executable attachment, but other types of files are equally dangerous
 - graphics, music, or photo images can contain code to be executed by an editor (macro virus)



Document (or macro) Viruses

- Implemented within a formatted document, such as a written document, a database, a slide presentation, a picture, or a spreadsheet
- These documents are highly structured files that contain both data (words or numbers) and commands (such as formulas, formatting controls, links).
 - The commands are part of a rich programming language, including macros, variables and procedures, file accesses, and even system calls.
 - The writer of a document virus uses any of the features of the programming language to perform malicious actions.



Autorun

- Each OS has files or programs automatically executed at the system startup or user login
 - Both system and utility programs
- Adding a malware to one of this programs make easy the activation and transmission



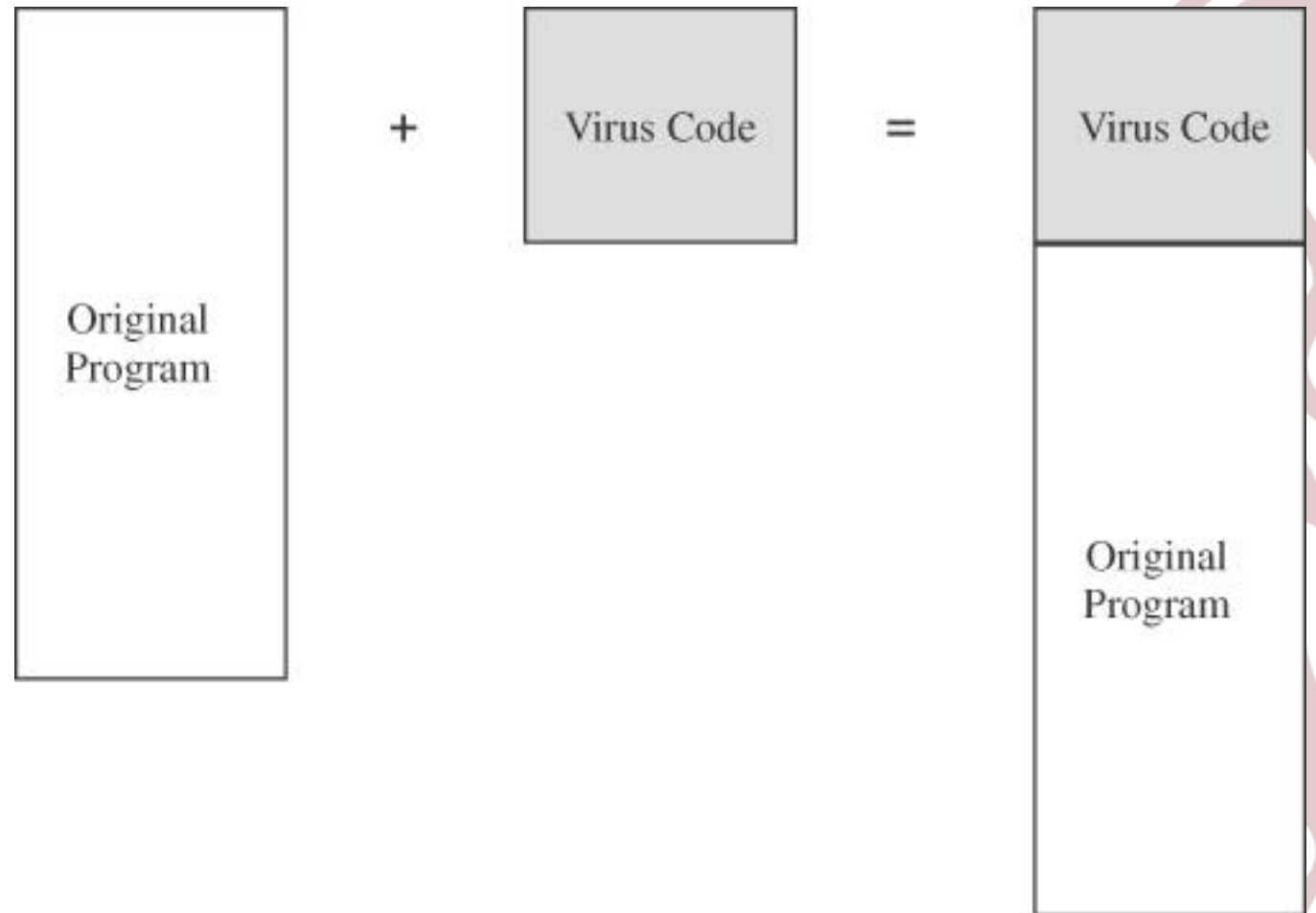
Propagation

A virus can be rather small, its code can be “hidden” inside other larger and more complicated programs

- Two hundred lines of a virus could be separated into 100 packets of two lines of code and a jump each
- These 100 packets could be easily hidden inside a compiler, a database manager, a file manager, or some other large utility

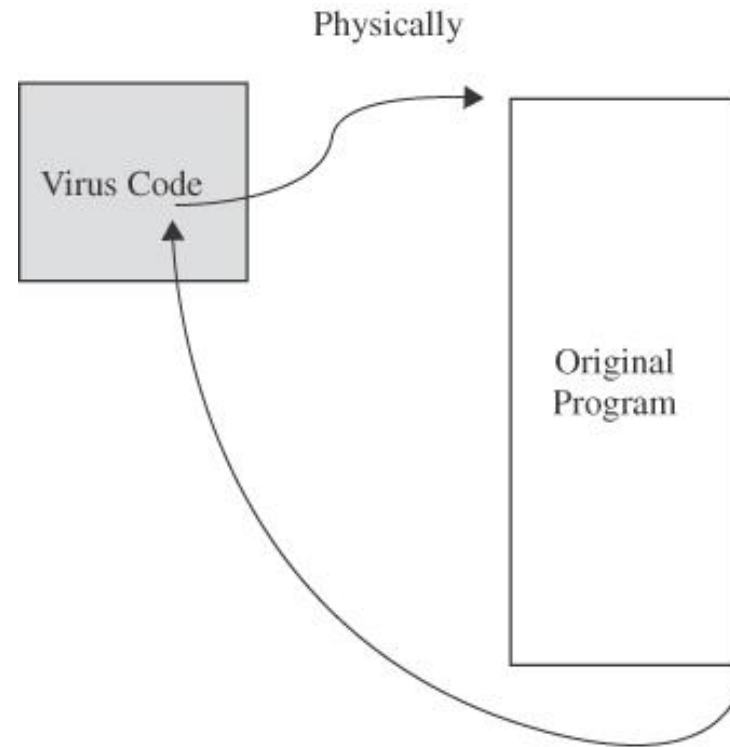


Appended Viruses



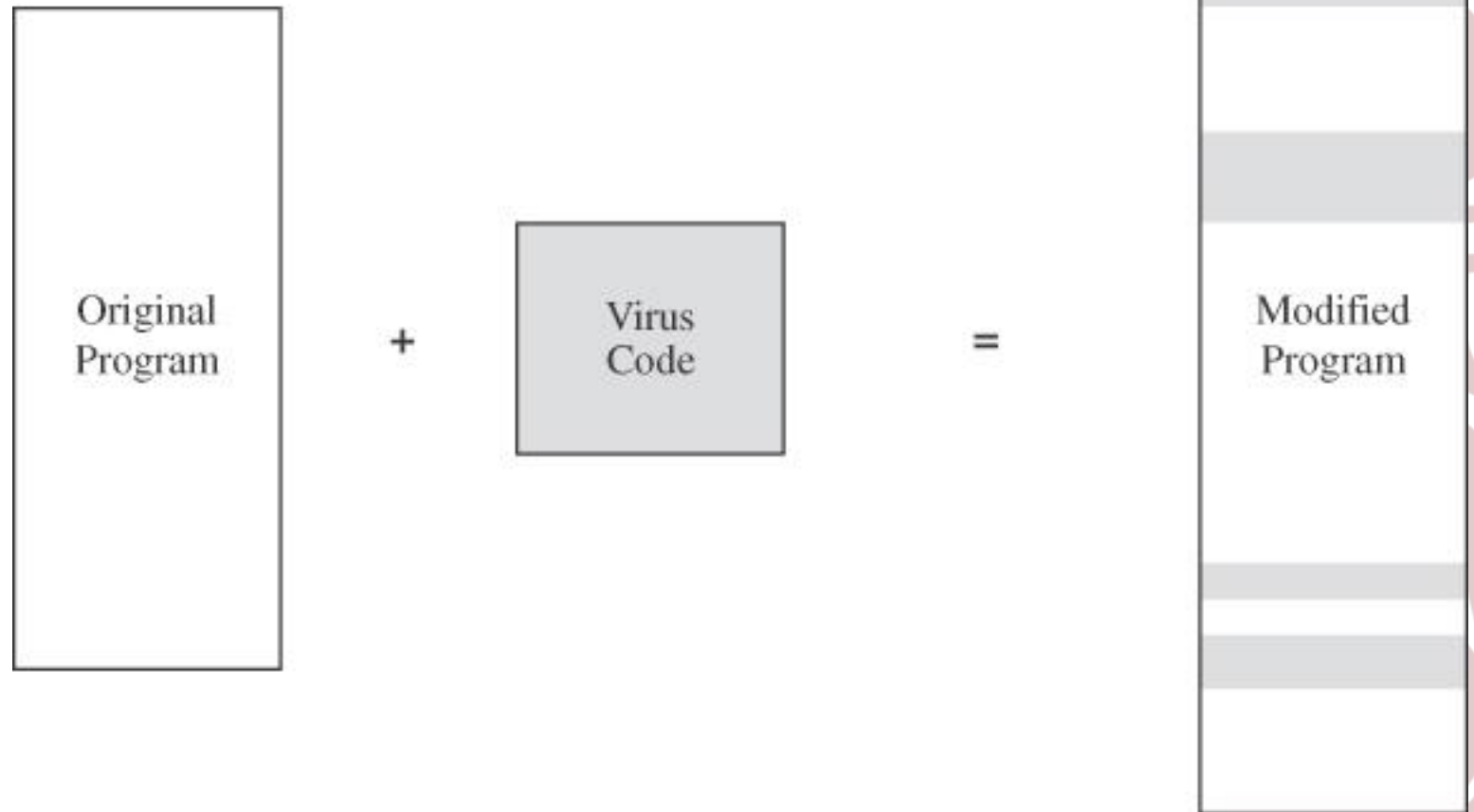
Virus that surround a program

- Can execute actions to modify the output of the program



Integrated Viruses and Replacements

- the virus writer has to know the exact structure of the original program to know where to insert which pieces of the virus
- the malicious code can replace an entire target, either mimicking the effect of the target or ignoring its expected effect and performing only the virus effect.



We are interested in exploring four aspects of malicious code infections:

- *harm*—how they affect users and systems
- *transmission and propagation*—how they are transmitted and replicate, and how they cause further transmission
- ***activation***—how they gain control and install themselves so that they can reactivate
- *stealth*—how they hide to avoid detection



Activation

The malware writer usually must find a vulnerability and hope the intended victim has not yet applied a protective or corrective patch

- Early malware writers used document macros and scripts as a vector for introducing malware into an environment
- Code Red worm of 2001 exploited an older buffer overflow program flaw in Microsoft's Internet Information Server (IIS)
- Conficker.A exploited a flaw involving a specially constructed remote procedure call (RPC) request



- Is it better to disclose a flaw and alert users that they are vulnerable or conceal it until there is a countermeasure?



How Malicious Code Gains Control

Malicious code such as a virus (V) has to be invoked instead of the target code that is really supposed to execute (T).

Essentially, the virus has three alternatives:

- has to seem to be T, saying effectively “I am T,”
- has to push T out of the way and become a substitute for T, saying effectively “Call me instead of T”
- can simply say “invoke me [you fool]”

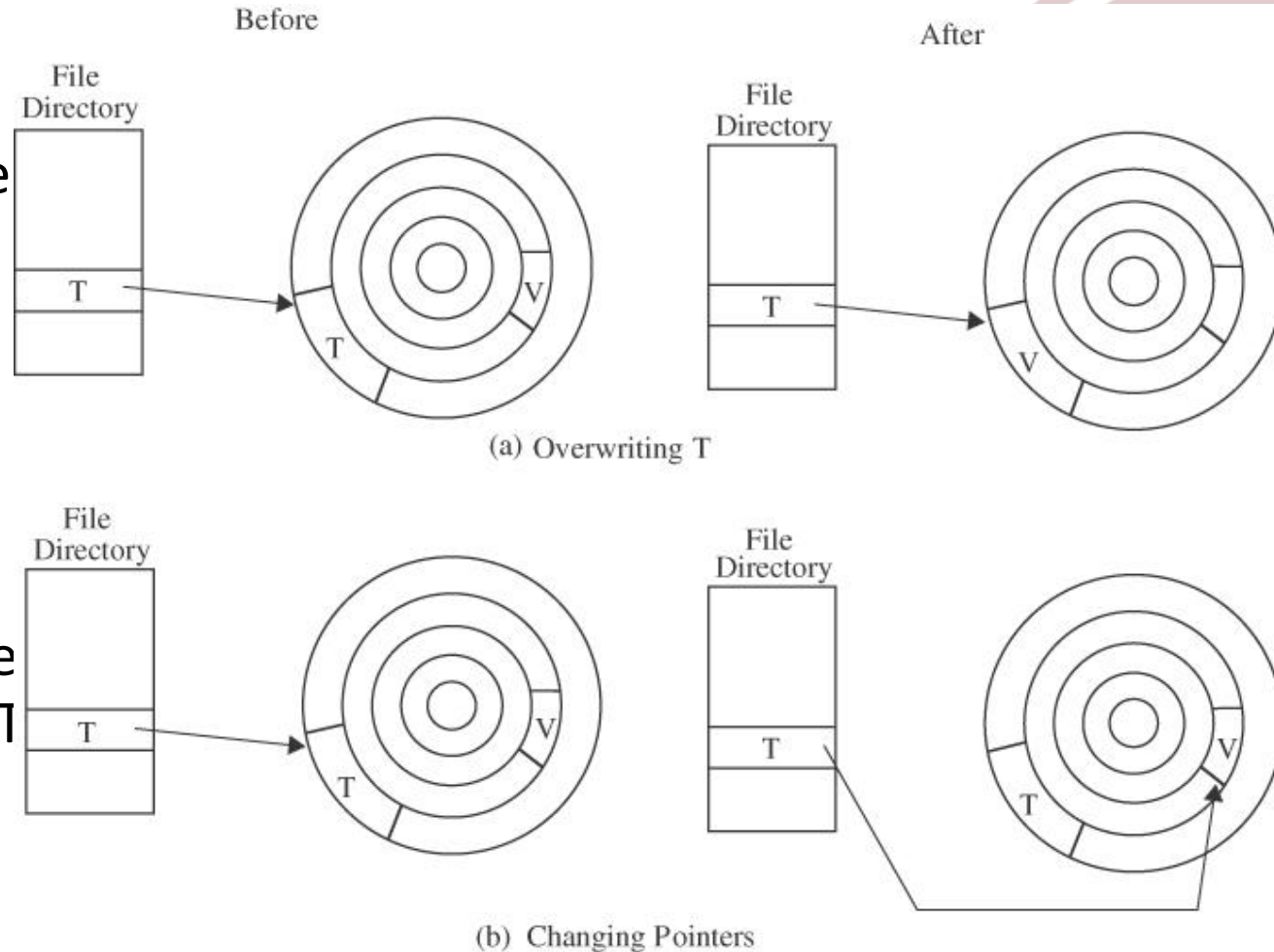


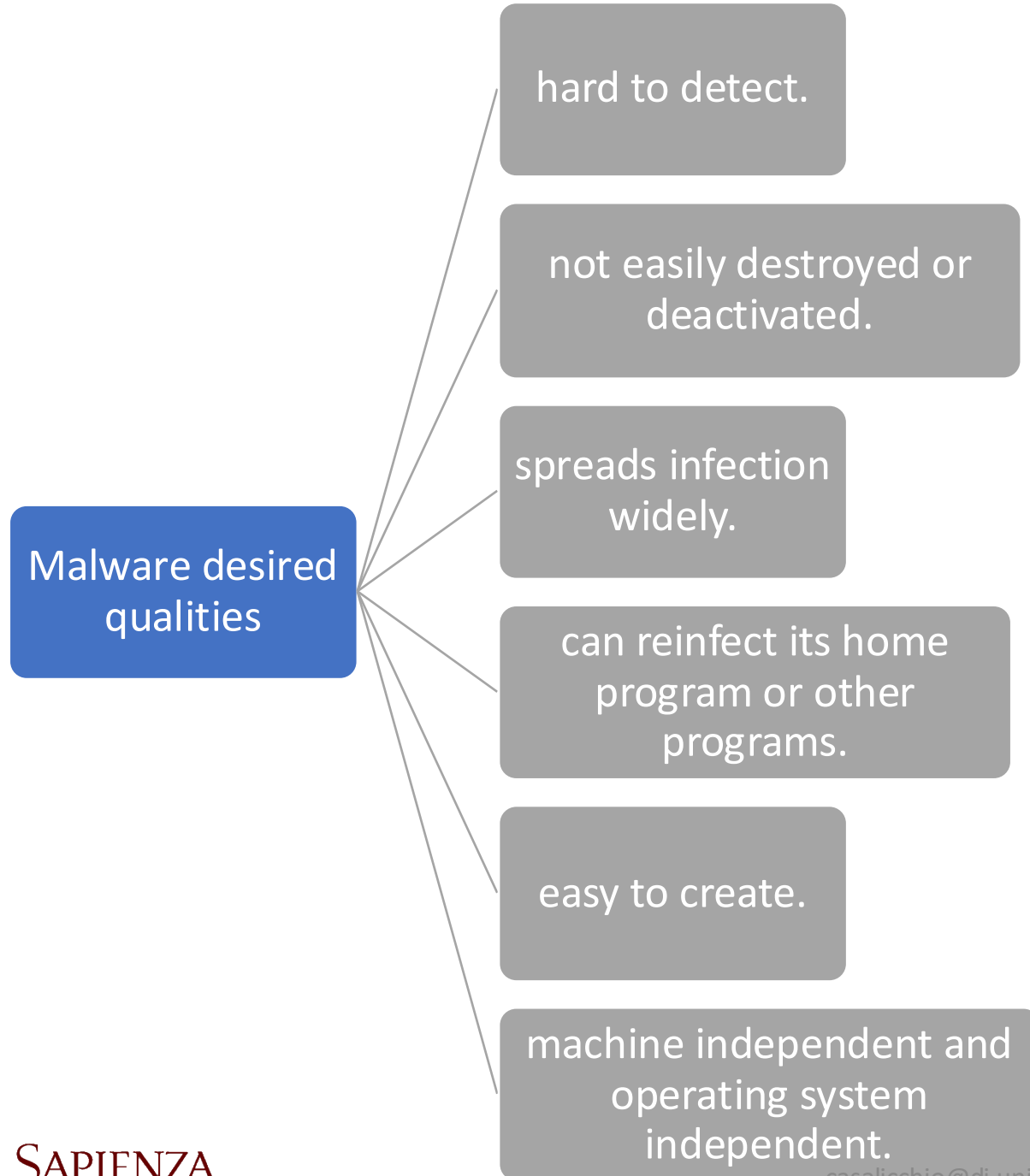
How a virus can become a program ready for activation

The virus can assume T's name by replacing (or joining to) T's code in a file structure.

There are two techniques

- to overwrite T in storage (simply replacing the copy of T in storage, for example).
- to change the pointers in the file table so that the virus is located instead of T whenever T is accessed through the file system.





Not all these qualities are required

- It depends on what the malware will do and where it will reside
- Examples
 - The virus should execute repeatedly so that the virus could multiply
 - It is enough to execute the virus once because the spreading is achieved by sending messages from the victim machine

One-Time Execution (Implanting)

Malicious code often executes a one-time process to transmit or receive and install the infection

- the user clicks to download a file
- the user opens an attachment
- the malicious code is downloaded silently as a webpage is displayed

This first step to acquire and install the code must be quick and not obvious to the user.



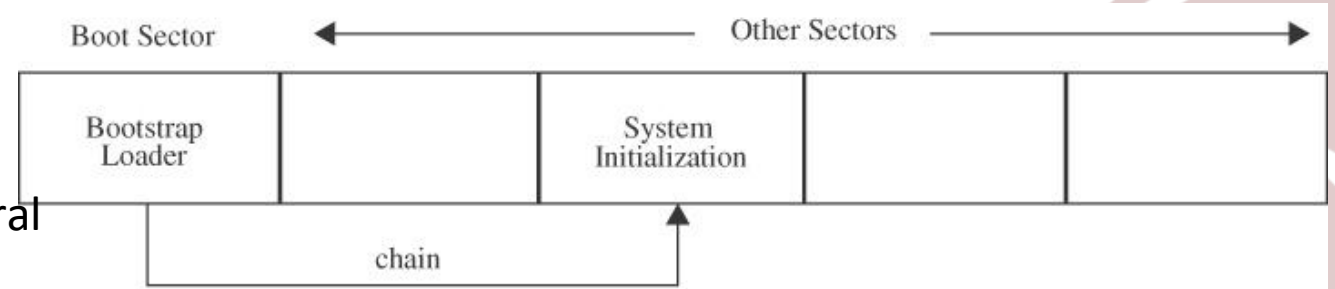
Boot Sector Viruses

Attacker goal

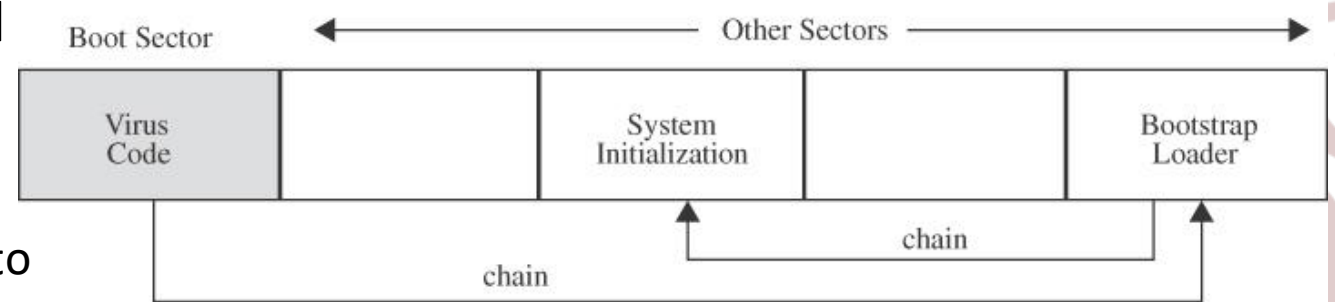
- creating continuing or repeated harm instead of just a one-time assault
 - The infection needs to become an integral part of the OS; reloaded each time the system is restarted

The bootstrap sequence is the target

- add to the list of modules to be loaded the malicious code
- substitute an infected module for a good one
 - by changing the address of the module to be loaded or
 - by substituting a modified routine of the same name



(a) Before infection



(b) After infection

Why the Boot Sector?

- The virus gains control before most detection tools are active, so that it can avoid, or at least complicate, detection.
- The files in the boot area are crucial parts of the operating system
 - Are made “invisible” by the OS preventing their deletion. Thus, the virus code is not readily noticed by users.



Resident code and Memory-Resident Viruses

Resident code: the code of frequently used parts of the operating system and a few specialized user programs

- e.g.
 - the routine that interprets keys pressed on the keyboard
 - the code that handles error conditions that arise during a program's execution
 - a program that acts like an alarm clock

Such “terminate and stay resident” routines are an appealing target to attach viruses because it is activated many times while the machine is running - Each time the resident code runs, the virus does too



Memory-resident viruses

- A boot sector virus might attach itself to a piece of resident code.
 - Then, each time the virus was activated, it might check whether any attached device was infected and, if not, infect it
- A virus can modify the operating system's table of programs to run
 - Once the virus gains control, it can insert a registry entry so that it will be reinvoked each time the system restarts.
 - Even if the user notices and deletes the executing copy of the virus from memory, the system will resurrect the virus on the next system restart
- By contrast, general malware executing just once from memory has only one opportunity to cause malicious behavior.
 - by disappearing whenever the machine is shut down, the malicious code is less likely to be analyzed by security teams



Viruses in non executable file

PDF, images, music files contains **interpretive data** used by the handler program (**the interpreter**) to perform actions on the file data

- E.g. Adobe Reader
 - If there is a flaw in the PDF interpreter or the semantics of the PDF interpretive language, opening a PDF file can cause the download and execution of malicious code.
 - So even an apparently passive object like a document image can lead to a malicious code infection.



Macro Virus

- Many applications, such as word processors and spreadsheets, have a “macro” feature
 - a user can record a series of commands and then repeat the entire series with one invocation
 - Such programs provide a “startup macro” that is executed every time the application is executed
- A virus writer can create a virus macro that
 - adds itself to the startup directives for the application.
 - then embeds a copy of itself in data files so that the infection spreads to anyone receiving one or more of those files.



```
macro Document_Open
  disable Macro menu and some macro security features
  if called from a user document
    copy macro code into Normal template file
  else
    copy macro code into user document being opened
  end if
  if registry key "Melissa" not present
    if Outlook is email client
      for first 50 addresses in address book
        send email to that address
        with currently infected document attached
      end for
    end if
    create registry key "Melissa"
  end if
  if minute in hour equals day of month
    insert text into document being opened
  end if
end macro
```

Figure 6.1 Melissa Macro Virus Pseudocode

We are interested in exploring four aspects of malicious code infections:

- *harm*—how they affect users and systems
- *transmission and propagation*—how they are transmitted and replicate, and how they cause further transmission
- *activation*—how they gain control and install themselves so that they can reactivate
- ***stealth***—how they hide to avoid detection



Stealth

Avoiding detection during **installation**, while **executing**, or even at **rest in storage**

- Most viruses maintain stealth by concealing their action, not announcing their presence, and disguising their appearance

Detection:

- It is about determining whether two programs are equivalent – the malicious and the checked one
 - It is an undecidable problem
 - Two modules may be equivalent, but produce subtly different results that may be security relevant



Stealth in Storage

- A malware is built to be distributed in many copies (like any general purpose software)
- Therefore, all copies should be identical; **that facilitate the detection**
 - Antivirus and other malicious code scanners look for pattern on the disk



Stealth in Storage (cont'd)

- The malware developer “solution” is to change the appearance of the malicious code, e.g.,
 - Rearrange the order of modules.
 - Rearrange the order of instructions (when order does not affect execution).
 - Insert instructions, (such as $A := A$), that have no impact.
 - Insert random strings (perhaps as constants that are never used).
 - Replace instructions with others of equivalent effect, such as replacing $A := B - 1$ with $A := B + 1(-1)$ or $A := B + 2 - 1$.
 - Insert instructions that are never executed (for example, in the *else* part of a conditional expression that is always true)



Introduction of Malicious Code



Introducing malicious code vs detection

- Insert directly the malicious executable code in an executable file (as user, system owner, administrator, or any authorized entity)
 - Prevention: scanning files; do not execute executable files from untrusted sources
- Insert the virus in an non executable file, that will be executed by the handler application
 - Prevention: scanning files; do not download/accept/open files from untrusted sources
- Using a MS feature to “hide” the executable file nature
 - The true document type is hidden in a field at the start of the file
 - The OS first try to open the file with the handler application associated with the extension (e.g. .pdf/pdf reader, .docx/msword, etc..) then if it fails with the application associated with the true file type (e.g., .exe)
 - Prevention: as above



Introducing malicious code vs detection (cont'd)

- **Steganography** (which means hiding in plain sight)
 - hiding malicious code in images or large dataset files
 - E.g., changing each 16th pixel of an image at high resolution, putting 1 bit of the virus code in a pixel, is not detectable by humans and makes hard the scanning

Approaches to introduce a virus/malware in a system are designed to make hard the detection

Execution patterns vs detection

Virus Effect	How It Is Caused
Attach to executable program	• Modify the file directory • Overwrite (replace) the executable program file
Attach to data or control file	• Modify directory • Rewrite data • Append to data • Append data to self
Remain in memory	• Intercept interrupt by modifying interrupt handler address table • Load self in nontransient memory area

Virus Effect	How It Is Caused
Infect disks	• Intercept interrupt or intercept operating system call (to format disk, for example) • Modify system file or ordinary executable program
Conceal self	• Intercept system calls that would reveal self • Classify self as “hidden” file
Spread infection	• Infect boot sector • Infect system program, ordinary program, or data that controls execution
Prevent deactivation	• Activate before antivirus program and subsequently block deactivation • Store copy to reinfect after deactivation

Many of these behaviors are perfectly normal and might otherwise go undetected

Transmission (diffusion) Patterns

Are many

- Boot sector → execution → executable file / data file
- I/O device → execution → boot sector
- ... → Execution → Network → ...

There is no specific/standard transmission pattern that can be used for detection



Polymorphic viruses

- The virus signature may be the most reliable way for a virus scanner to identify a virus
- Signature an invariant pattern in the code, e.g.
 - always begins with the string 0x47F0F00E08 and has string 0x00113FFF located at word 12
- Hiding by changing the form of the code and reducing the probability a signature match (or does exist)

Polymorphic virus (poly means “many” and morph means “form”) a virus that can change its appearance.



How many form?

- **Two-form** polymorphic can be handled easily as two independent viruses
 - the virus writer will want either a large or an unlimited number of forms so that the number of possible forms is too large for a virus scanner to search for
- **Many forms:** a polymorphic virus has to randomly reposition all parts of itself and randomly change all fixed data
 - E.g., instead of containing the fixed (and therefore searchable) string “HA! INFECTED BY A VIRUS,” a polymorphic virus has to change even that pattern sometimes
- Trivial example: assume a virus writer has 100 bytes of code and 50 bytes of data to implant. To make two virus instances different, the writer might:
 - distribute the first version as 100 bytes of code followed by all 50 bytes of data
 - A second version could be 99 bytes of code, a jump instruction, 50 bytes of data, and the last byte of code.
 - Other versions are 98 code bytes jumping to the last two, 97 and three, and so forth.

Encrypting viruses

A simple variety of polymorphic virus **uses encryption under various keys to make the stored form of the virus different**

Composed of three parts:

- a decryption key
- the (encrypted) object code of the virus
- the (unencrypted) object code of the decryption routine

The decryption routine itself or a call to a decryption library routine must be in the clear, and so that becomes the signature

To avoid detection, not every copy of a polymorphic virus must differ from every other copy



Countermeasures

For users

For developers (SE techniques, Security specific techniques, other useless techniques)



User Vigilance

- Use only commercial software acquired from reliable, well-established vendors
- Test all new software on an isolated computer
- Open attachments—and other potentially infected data files—only when you know they are safe
- Install software—and other potentially infected executable code files—only when you really, really know they are safe
- Recognize that any website can be potentially harmful
- Make a recoverable system image and store it safely
- Make and retain backup copies of executable system files
- Apply all patches promptly except when doing so would cause more harm than good
- Use malware scanners

**Recommendation are not an enforcement, we
cannot rely on user due diligence**



Virus Detection

Virus scanners are tools that look for signs of malicious code infection. Most such tools look for a signature or fingerprint, a telltale pattern in program files or memory.

Limitations

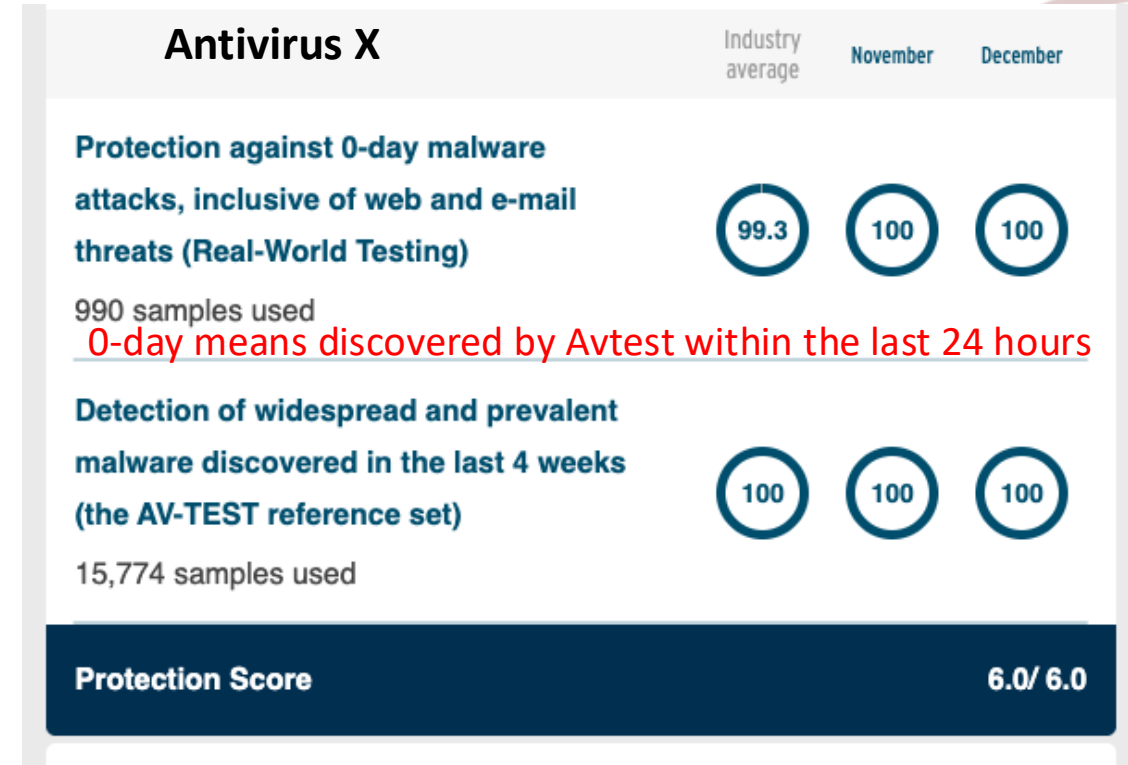
- are necessarily retrospective, looking for patterns of known infections – new viruses pass undetected
 - Malware writers have a brief time window during which the strain's pattern will not be in the scanner database
- patterns are necessarily static (start with, contains); longer the pattern more precise the classification (necessary to avoid misclassifications)
 - Malicious code writers are conscious of pattern matching, so they vary their code to reduce the number of repeated patterns



Note

- The book report that in 2014 “Symantec’s Senior Vice President for Information Security admitted that antivirus software catches only 45% of malicious code”
- Today the detection rate is close to 100% at least for what AVtest reports

<https://av-test.org/en/antivirus/home-windows/>



The test set is very small considering the number of virus subspecies around the internet (billions)

A note on the concept of Zero-day

Zero-day vulnerability; Zero-day attack

- Originally the concept referred to the time of release of the software
- More usually refers to the time the vulnerability is discovered
 - Discovered by the software producer or white hat or black hat
- Since the day-0 there is a short time window the attacker can leverage the zero-day vulnerability

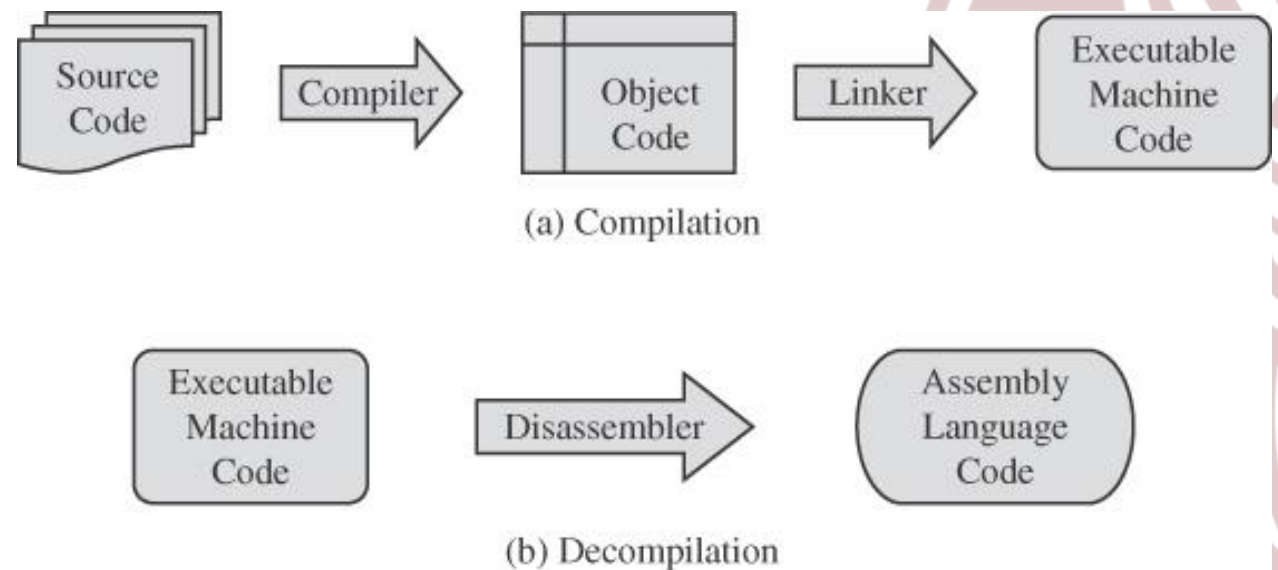


Virus detection: Code Analysis

An approach to detecting an infection by analyzing the code to determine **what it does**, **how it propagates**, and perhaps even **where it originated**.

A cybersecurity expert can start analyzing the in memory resident part of the running virus, but that will contain only part of the whole malware code

He/She has **to look at the static code**



Not an easy job, the outcome depends on the expert knowledge, experience, ingenuity

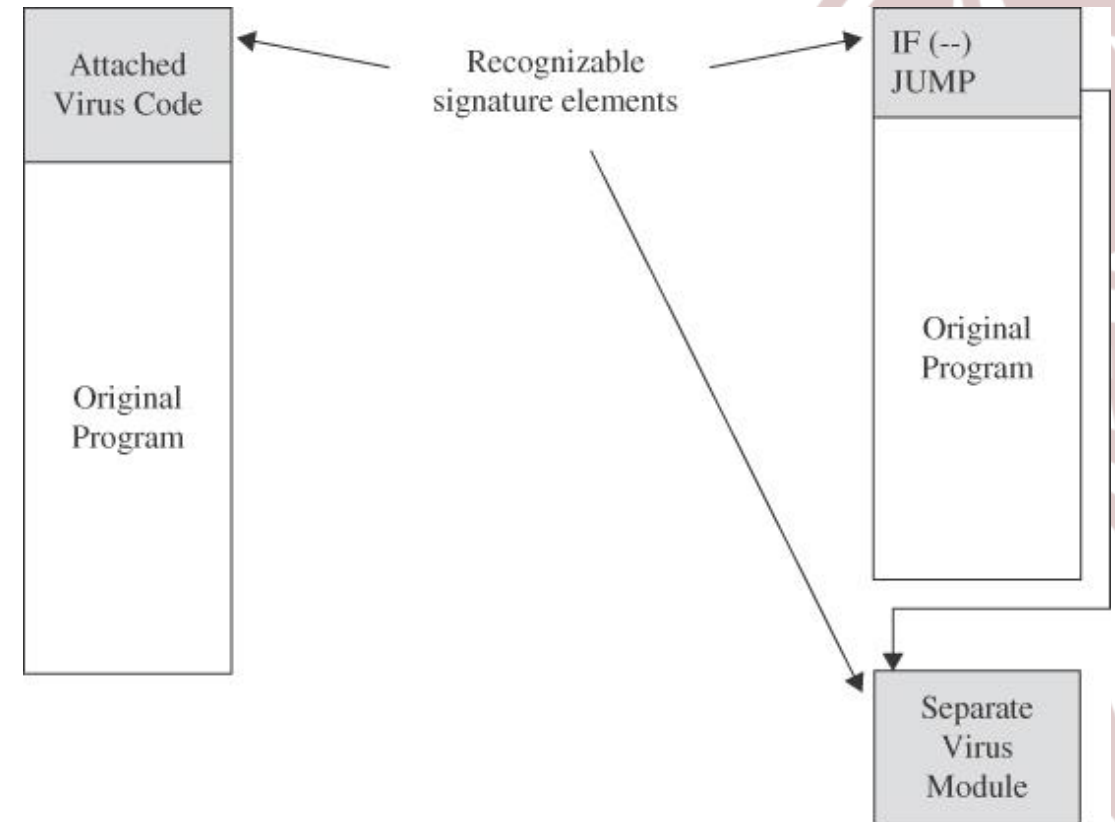
Virus Detection: Storage Patterns

Detectable effects of infection

- Larger Filesize with respect the original file
- Same size, but code overwritten by the malware code (not all functionalities works)

The virus scanner can

- use a code or checksum to detect changes to a file
- look for suspicious patterns, such as a Jump instruction as the first instruction of a system program



Countermeasures

For users

For developers (SE techniques, Security specific techniques, other useless techniques)



Countermeasures for developers

- Software engineering techniques that have been shown to improve the security of code
- Security specific techniques
- Techniques that don't work

Ideally, developers should have a reasonable understanding of security, and especially of thinking in terms of threats and vulnerabilities

- Armed with that mindset and good development practices, programmers can write code that maintains security



Countermeasure: Software Engineering Techniques

Modularity

Encapsulation

Information Hiding

Mutual Suspicion

Confinement

Simplicity

Genetic Diversity

Testing



Modularity

A key principle of SE is **modularity** - To design the SW as small, self-contained units called components or modules

Advantages in term of security

- Flaws remains isolated
- Easy maintainability
- Simpler Testing
- Easy vulnerability detection with encapsulated (isolated) components
- Information hiding



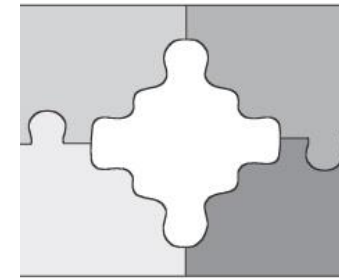
Modularity – four conditions

The goal is for each component to meet four conditions:

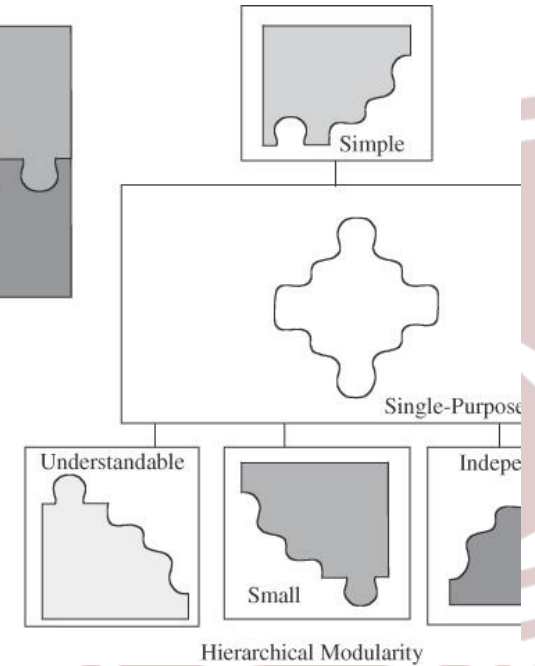
- **single-purpose**, performs one function
- **small**, consists of an amount of information for which a human can readily grasp both structure and content
- **simple**, is of a low degree of complexity so that a human can readily understand the purpose and structure of the module
- **independent**, performs a task isolated from other modules

Other component characteristics indicating modularity

- having a single entry and single exit flow
- using a limited set of programming constructs



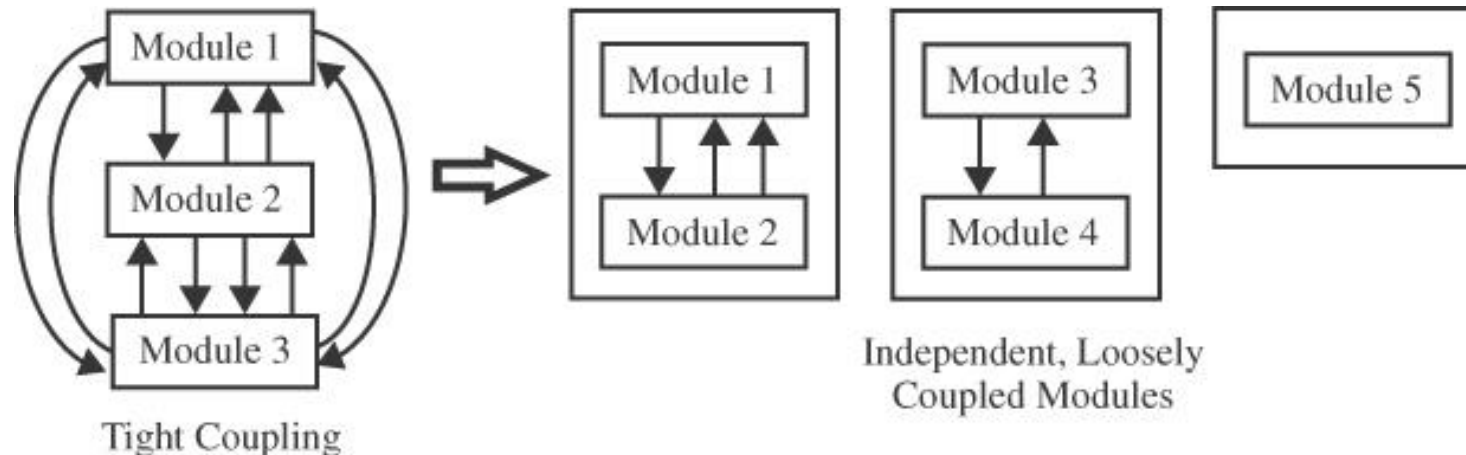
Single
Monolithic
Structure



Modularity: Cohesion and Coupling

Modular components have **high cohesion** and **low coupling**

- By **cohesion**, we mean that all the elements of a component have a logical and functional reason for being there
- **Coupling** refers to the degree with which a component depends on other components in the system



Advantages of modularity

General

- Maintenance is easier
- Understandability is higher
- Reuse increase
- Correctness is easier to evaluate
- Testing improves by reducing cost and time

Security

- Flaws remains isolated
- Easy maintainability
- Simpler Testing
- Easy vulnerability detection with encapsulated (isolated) components
- Information hiding

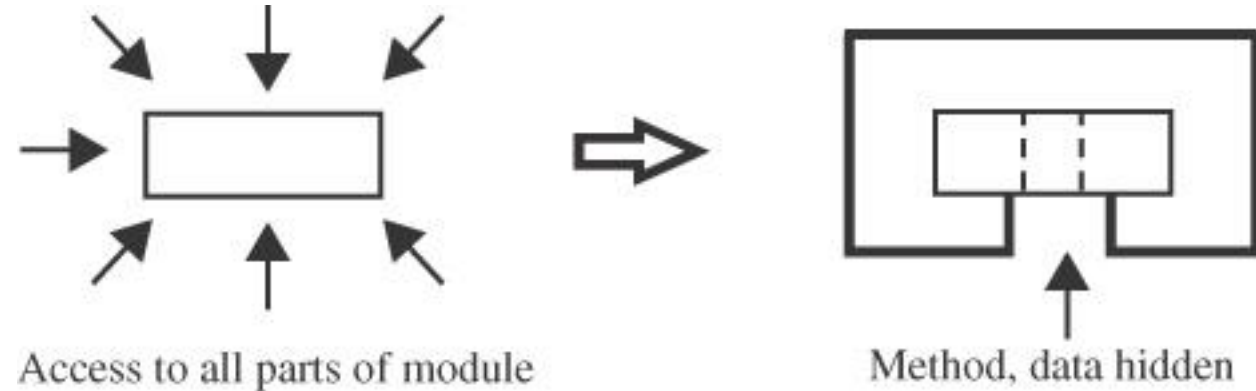
Encapsulation

Encapsulation is a technique for packaging the information [inside a component] in such a way as to **hide what should be hidden and make visible what is intended to be visible**

- **Sharing** is
 - carefully documented so that a component is affected only in known ways by other components in the system.
 - minimized so that the fewest interfaces possible are used.



Information Hiding



- We can think of a component as a black box, with certain well-defined inputs and outputs and a well-defined function
 - Other components' designers do not need to know how the module completes its function; it is enough to be assured that the component performs its task in some correct manner
 - Information hiding is desirable because malicious developers cannot easily alter the components of others if they do not know how the components work
- the security of the module should not depend only on its structure remaining hidden.

Mutual Suspicion

Mutually suspicious programs operate as if other routines in the system were malicious or incorrect

- A calling program cannot trust its called subprocedures to be correct, and a called subprocedure cannot trust its calling program to be correct
- Each protects its interface data so that the other has only limited access

E.g., a procedure to sort the entries in a list cannot be trusted not to modify those elements, while that procedure cannot trust its caller to provide any list at all or to supply the number of elements predicted

The implication is that each procedure and subprocedure will introduce mechanisms such as input/output verification and exception/error handling

Confinement

Limiting the resources a program can access

Strong confinement would be particularly helpful in limiting the spread of viruses.

- Since a virus spreads by means of transitivity and shared data, all the data and programs within a single compartment of a confined program can affect only the data and programs in the same compartment.



Simplicity

Simple solutions are easier to understand, leave less room for error, and are easier to review for flaws

“Complexity is often the enemy of security.” —Amit Yoran, RSA



Genetic Diversity

Having many components of a system come from one source or relying on a single component is risky

Diversity creates a moving target for the adversary

- Introducing diversity automatically is possible
 - A compiler can generate different but functionally equivalent object code from one source file; helps protect one version from harm that might affect another version.

Tight integration of products is a similar concern - A vulnerability in one subsystems can affect some or all of the others, e.g.,

- Windows linked to Internet Explorer, the Edge browser, the Office suite, Outlook, OneDrive
- Android linked to Google functions and the Google cloud.



Unit Testing

- Done in a controlled environment; verify the component works as specified

Integration Testing

- verifying that the system components work together as described in the system and program design specifications

Function Testing

determine whether the integrated system comply with requirement specifications (**including Security requirements**)

Performance Testing

compares the system with the remainder of software and hardware requirements (**including Security requirements**)

Acceptance Testing

the system is checked against the customer's requirements description

Installation Testing

to make sure that the system still functions as it should **“and nothing more” for security requirements**

Regression Testing

After an enhancement or problem fix that all remaining functions are still working and performance has not been degraded



```
if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
    goto fail;
if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
    goto fail;
    goto fail;
if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
    goto fail;

...

fail:
    SSLFreeBuffer(&signedHashes);
    SSLFreeBuffer(&hashCtx);
    return err;
```

C'e' qualche bug in
questo codice?

The GOTO Fail bug (February 2014, Apple iOS operating system)

- If the first two conditional statements are false, execution drops directly to the duplicate “goto fail” line and exits the routine
- **err is 0** because the SSLHashSHA1.update call will generally not return an error; the value of err will almost always be zero when the second goto fail statement is executed
- The return value of zero is provided to the caller, who believes that the signature verification on the “Server Key Exchange” message passed
 - The impact of this flaw is that even insecure web connections are treated as secure.

```
fail:  
    SSLFreeBuffer(&signedHashes);  
    SSLFreeBuffer(&hashCtx);  
    return err;
```

```

static OSStatus
SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa, SSLBuffer signedParams,
                                uint8_t *signature, UInt16 signatureLen)
{
    OSStatus      err;
    ...

    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;

    // code omitted for brevity...

    err = sslRawVerify(ctx,
                      ctx->peerPubKey,
                      dataToSign,          /* plaintext */
                      dataToSignLen,       /* plaintext length */
                      signature,
                      signatureLen);

    if(err) {
        sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify "
                   "returned %d\n", (int)err);
        goto fail;
    }

fail:
    SSLFreeBuffer(&signedHashes);
    SSLFreeBuffer(&hashCtx);
    return err;
}

```

Oops...

Never gets called (but needed to be)...

Despite the name, always returns "it's OK!!!"

Parameters control
part

this is the method
that actually
checks the
certificate validity

function returns 0 without doing the
certificate verification work, simply
returning an optimistic "everything's
OK!" back to the caller

<https://www.zdnet.com/article/apples-goto-fail-tells-us-nothing-good-about-cupertinos-software-delivery-process/>



Testing perspectives

Black-box testing

- testers cannot “see inside” the system, so they apply particular inputs and verify that they get the expected output

Clear-box testing

- testers can examine the design and code directly, generating test cases based on the code’s actual construction.
- E.g., clear-box testing reveals that component X uses CASE statements and can look for instances in which the input causes control to drop through to an unexpected line.



Key ingredients for testing

Product expertise

- The tester needs to understand the requirements and functionality of the object being tested
- The tester should have sufficient familiarity with the product to be able to predict what it cannot do and be able to stress it in all its configurations

Coverage

- Testing must be complete, in that no component should be ignored, no matter how small or insignificant

Risk analysis

- Testing can never cover everything. It is important to spend testing resources wisely and effectively
- Risk analysis answers two key questions: **What are the most critical pieces? What can go seriously wrong?** The answers form the basis for the testing's priorities

Domain expertise

- A tester must understand the product being tested. Trivially, someone cannot effectively test a Fahrenheit-to-Celsius converter without understanding those two temperature scales



Key ingredients for testing (cont'd)

Common vocabulary

- There is little common vocabulary for testing; even terms like “black-box testing” are subject to some interpretation. More important, testers need to be able to share patterns and techniques with one another, and to do that, they need some common understanding of the larger process.

Variation

- Testing is not a checklist exercise; if it were, we would automate the whole process, confirm the results, and eliminate product failures.
- Testers need to vary their routines, test different things in different ways, and adapt to successes and failures.

Boundaries

- Because testing can continue indefinitely, some concept of completeness and sufficiency is necessary
- Finite resources of time or money dictate how much testing is done
- A better approach is a rational plan that determines what degree of testing is adequate based on perceived risk.



Limitations of Testing

Passing tests does not demonstrate the absence of failure

- Can only demonstrate the *existence* of a failure

Time or effort constraints;

- the combinatorial explosion of both inputs and internal states makes complete testing complex and time consuming

Does not ensure any degree of completeness

- Sometimes focuses only on observable effects, not the internal structure of a product

Testing the internal structure of a product involves modifying the product by adding code to extract and display internal states

- That extra functionality affects the product's behavior and can itself **be a source of vulnerabilities or can mask other vulnerabilities.**



Countermeasure Specifically for Security

- Security Design Principles
- Penetration Testing for Security
- Proofs of Program Correctness
- Validation
- Defensive Programming



Security by design principles (Saltzer & Schroeder)

Least privilege.

- Each user and each program should operate using the fewest privileges possible. In this way, damage from an inadvertent or malicious attack is minimized.

Economy of mechanism.

- The design of the protection system should be small, simple, and straightforward. Such a protection system can be carefully analyzed, exhaustively tested, perhaps verified, and relied on.

Open design.

- The protection mechanism must not depend on the ignorance of potential attackers; the mechanism should be public, depending on secrecy of relatively few key items, such as a password table. An open design is also available for extensive public scrutiny, thereby providing independent confirmation of the design security.

Complete mediation.

- Every access attempt must be checked. Both direct access attempts (requests) and attempts to circumvent the access-checking mechanism should be considered, and the mechanism should be positioned so that it cannot be circumvented.

Security by design principles (cont'd)

Permission based.

- The default condition should be denial of access. A conservative designer identifies the items that should be accessible rather than those that should not.

Separation of privilege.

- Ideally, access to objects should depend on more than one condition, such as user authentication plus a cryptographic key. In this way, someone who defeats one protection system will not have complete access.

Least common mechanism.

- Shared objects provide potential channels for information flow. Systems employing physical or logical separation reduce the risk from sharing.

Ease of use.

- If a protection mechanism is easy to use, it is unlikely to be avoided.



Top 10 Secure Coding Practices

1. Validate input.
2. Heed compiler warnings.
3. **Architect and design for security policies.**
4. **Keep it simple.**
5. **Default to deny.**
6. **Adhere to the principle of least privilege.**
7. Sanitize data sent to other systems.
8. **Practice defense in depth.**
9. Use effective quality-assurance techniques.
10. Adopt a secure coding standard.

4,5,6 match Saltzer &
Schroeder principles
3,8 naturally derived from
them



Penetration testing (or tiger team analysis or ethical hacking)

- it involves the use of a team of experts trying to crack the system being tested (as opposed to trying to break into the system for unethical reasons)
- A system that fails penetration testing is known to have faults; one that passes is known only not to have the faults tested for
 - penetration testing is useful and often finds faults that might have been overlooked by other forms of testing
- Pen-testing carefully a system could require months not weeks



Proofs of Program Correctness

Program verification can demonstrate formally the “correctness” of certain specific programs

- It involves making **initial assertions about the program’s inputs** and then checking to see whether the desired output is generated
- **Each program statement is translated into a logical description** about its contribution to the logical flow of the program
- The **terminal statement of the program is associated with the desired output.**
- By applying a logic analyzer, **we can prove that the initial assumptions, plus the implications of the program statements, produce the terminal condition.**



Limitations of program verification

- Correctness proofs depend on a programmer's or logician's ability to translate a program's statements into logical implications
- Deriving the correctness proof from the initial assertions and the implications of statements is difficult
- Too often people focus so much on the formalism and on deriving a formal proof that they ignore the underlying security properties to be ensured
- Program verification systems are being improved constantly. Larger programs are being verified in less time than before. Today it is an important control to assure the security of programs



Validation

Is the counterpart to verification, substantiating that the system developers have implemented all requirements

- **Validation** makes sure that the implementation is building according to the specification, and
- **Verification** checks the quality of the implementation.

Validation approaches

- Requirements checking
 - cross-check each system requirement with the system's source code or execution-time behavior
 - demonstrates only that the system does everything it should do (not what should not do)
- Design and code reviews
 - address system correctness and requirement implementation
- System testing
 - behaviors and data expected from reading the requirements document can be confirmed in the actual running of the system



Defensive programming

Defenders have to counter all possible attacks, whereas attackers only need to find one weakness to exploit. Thus, a strong defense is not only helpful, it is essential.

- Program designers and implementers must write correct code but also anticipate what could go wrong, e.g. handling of inappropriate/incorrect input data

“Secure software must be able to continue operating as expected even when under attack. So a software system’s security is neither a specific library or function call nor an add-on that magically transforms existing code. A system’s security is the holistic result of a thoughtful approach applied by all stakeholders throughout the software development life cycle”



Countermeasures that don't work

- penetrate-and-patch
- security by obscurity



Penetrate-and-patch

- A tiger team test a system's security by attempting to cause it to fail
- The test was considered to be a proof of security. Unfortunately, far too often the attempted proof instead became a process for generating counterexamples, in which not just one but several serious security problems were uncovered
- The patch efforts were largely useless, generally making the system *less* secure because they frequently introduced new faults even as they tried to correct old ones

Penetrate-and-patch fails because it is hurried, misses the context of the fault, and focuses on one failure, not the complete system.



Security by Obscurity

- Security by obscurity is the belief that a system can be secure as long as nobody outside its implementation group is told anything about its internal mechanisms
- Many examples show that approach to hide data really don't- so **things meant to stay hidden seldom do. Attackers find and exploit many hidden things**
- Late in 19th century A. Kerckhoffs said: **“The system must not depend on secrecy, and security should not suffer if the system falls into enemy hands”**

